📋 შემაჯამებელი კითხვარი

პროგრამა: Front-end პროგრამირება

მოდული: პრაქტიკული პროექტი (06.1.1.9)

სტუდენტი: იობი შარაშენიძე

პროექტი: WMS — საწყობის მართვის სისტემა

GitHub: https://github.com/Iobis21/finally-project

Live Demo: https://finally-project-3lf1.vercel.app

ბლოკი I: პროექტის დაგეგმვა, რისკები და ტექნოლოგიური სტეკი

1. სამუშაო გეგმა და Milestones

ეტაპებად დაყოფა საშუალებას იძლევა, თავიდანვე დავინახოთ სამუშაოს მოცულობა, თანმიმდევრობა და ვადები. ჩემს პროექტში 4 ეტაპი იყო: დაგეგმვა და არქიტექტურა → UI და Layout → CRUD, State და ავტორიზაცია → ოპტიმიზაცია, ტესტირება და დოკუმენტაცია. თასქებს GitHub Projects-ის კანბან დაფაზე Story Points-ით ვმართავდი. დაგეგმვის გარეშე კოდის წერისას ხშირია ქაოტური სტრუქტურა, განმეორებითი გადაკეთება და ვადების გადაცილება, რადგან დამოკიდებულებები (მაგ. როლები → დაცული გვერდები) დროულად არ ჩანს.

2. რისკების ანალიზი

რისკი 1: მონაცემების დაკარგვა გვერდის დარეფრეშებისას. პრევენცია: გლობალური Store სინქრონდება `localStorage`-თან, ხოლო სესია (JWT) ცალკე ინახება. **რისკი 2: ორმაგი დაჭერა და ერთდროული ჩამოწერა (Race Condition).** პრევენცია: გაგზავნისას ღილაკი ბლოკდება, Zod ამოწმებს `.max(ნაშთი)`-ს, ჩაწერამდე კი სერვერიდან უახლესი ნაშთი მოწმდება. **დროითი რისკი: დიდი სფერო ერთი დეველოპერისთვის.** პრევენცია: ჯერ გაკეთდა MVP (ავტორიზაცია, მარაგები, CRUD), მეორეხარისხოვანი მოდულები კი v2.0-ში გადავიდა.

3. ტექნოლოგიური სტეკის შერჩევა

კრიტერიუმები იყო: კურსის მოთხოვნები, ჩემი ცოდნის დონე, საზოგადოების მხარდაჭერა და შედეგის ხარისხი. ავირჩიე **React 19 + Vite** (კომპონენტები და სწრაფი build), **TypeScript** (ბევრი ტიპის მონაცემი, შეცდომები გაშვებამდე ჩანს), **Tailwind CSS + CSS ცვლადები** (დიზაინ-ტოკენები ერთ ადგილას), **Context API + useReducer** (გლობალური Store დამატებითი ბიბლიოთეკის გარეშე) და **React Hook Form + Zod** (ფორმები და ვალიდაცია). ბექენდისთვის ავიღე json-server და json-server-auth (JWT), რომ ფრონტენდზე კონცენტრაცია შემენარჩუნებინა.

ბლოკი II: არქიტექტურა, სემანტიკური HTML5 და Git

4. სემანტიკური მარკირება (HTML5)

გამოვიყენე `<header>` (Header), `<aside>` + `<nav aria-label>` (მენიუ), `<main>` (გვერდის შიგთავსი), `<section>` (სტატისტიკა, პროდუქტები), `<article>` (თითო ბარათი) და `<footer>`. ასევე ცხრილში `<th scope>`, ბარათებში კი `<dl>`. სემანტიკური თეგები

ეკრანის წამკითხველს (Screen Reader) ეხმარება გვერდის სტრუქტურის გაგებასა და ნავიგაციაში, საძიებო სისტემებს კი შიგთავსის სწორად ინდექსირებაში. შედეგად Lighthouse-ის Accessibility და SEO ქულა 100-ია.

5. მოდულური არქიტექტურა (SOLID / DRY)

საქაღალდეები დავყავი ფენებად: `components/` (მხოლოდ UI), `pages/` (გვერდები), `hooks/` (ლოგიკა), `store/` (გლობალური state), `services/` (ყველა HTTP მოთხოვნა), `types/` და `schemas/`. ამის წყალობით ერთი ფუნქცია ერთხელ იწერება და ბევრგან გამოიყენება (DRY): `request()` ყველა API მოთხოვნისთვის, `StatusBadge` სამ ადგილას, `FormField` ყველა ფორმაში. კომპონენტებში `fetch` არ არის, ამიტომ API-ს შეცვლისას მხოლოდ `services/` იცვლება (SOLID, Dependency Inversion).

6. Git და Branching

ცალკე `feature/` ბრენჩზე მუშაობისას `main` ყოველთვის სტაბილური რჩება. ცვლილებები Pull Request-ით მოწმდება და ერთიანდება, საჭიროების შემთხვევაში კი ადვილად უქმდება. ჩემს პროექტში ფინალური დოკუმენტაცია გაკეთდა `docs/final-readme` ბრენჩზე და შეუერთდა Pull Request #1-ით. Conventional Commits არის commit-ის შეტყობინების სტანდარტი, სადაც პრეფიქსი ცვლილების ტიპს აჩვენებს: `feat:` ახალი ფუნქციაა, `fix:` ბაგის გასწორება, `refactor:` კოდის გადაწყობა, `perf:` ოპტიმიზაცია, `docs:` დოკუმენტაცია. ჩემს რეპოზიტორიაში 70-მდე ასეთი ატომური commit-ია.

ბლოკი III: გლობალური სტილები, დიზაინ-სისტემა და Layout

7. CSS ცვლადები და დიზაინ-ტოკენები

გლობალურ ცვლადებში (`:root`, Tailwind `@theme`) ფერი, შრიფტი და დაშორება ერთხელ იწერება, ამიტომ მისი ერთ ადგილას შეცვლა მთელ საიტზე აისახება. ასე იზრდება დიზაინის თანმიმდევრულობა და შეცდომით „განსხვავებული ლურჯის“ რისკი ქრება. ჩემს პროექტში მაგალითებია `--primary`, `--space-md` (8px ბადე: 4/8/16/24/32), `--radius-md` და სტატუსების ფერები. ამავე ცვლადებით ადვილად კეთდება მუქი თემაც (`.dark`).

8. Flexbox vs CSS Grid

Flexbox ერთგანზომილებიანია: ალაგებს ელემენტებს ერთ ხაზზე (სტრიქონი ან სვეტი). CSS Grid ორგანზომილებიანია: მართავს სტრიქონებსა და სვეტებს ერთდროულად. ჩემს პროექტში Grid გამოვიყენე გვერდის ჩონჩხისთვის (`260px 1fr` + `grid-template-areas`: Header, Sidebar, Main, Footer) და ბარათების ბადისთვის (`repeat(auto-fit, minmax(220px, 1fr))`). Flexbox კი Header-ში (ლოგო მარცხნივ, პროფილი მარჯვნივ) და ბარათის შიგნით (იკონა + ტექსტი).

9. UI კომპონენტების ხელახალი გამოყენებადობა

უნივერსალური კომპონენტი ცვალებად ნაწილებს Props-ით იღებს, შიდა კოდის შეცვლის გარეშე. მაგალითად, `<Button variant="danger" isLoading>` ფერს variant-ით ცვლის, ხოლო `StatusBadge` თვითონ ითვლის სტატუსს გადაცემული `item`-დან. `Modal` შიგთავსს `children`-ით იღებს, ამიტომ ერთი და იგივე მოდალი გამოიყენება ფორმისთვის, ჩამოწერისთვის და წაშლის დადასტურებისთვის (SOLID, Open/Closed).

ბლოკი IV: ინტერაქტიულობა, მოდალები, Responsive და Debugging

10. ძიების ოპტიმიზაცია (useDebounce)

Debounce-ის გარეშე ძიება ყოველ ასოზე ეშვება („ლეპტოპი“ 7 ფილტრაციას გამოიწვევდა), რაც ზედმეტ გამოთვლასა და სერვერის მოთხოვნებს ნიშნავს. `useDebounce` ყოველ ცვლილებაზე იწყებს `setTimeout`-ს 300ms-ით, ხოლო ახალი ასოს აკრეფისას cleanup-ში `clearTimeout` ძველ ტაიმერს აუქმებს. ამიტომ ძიება მხოლოდ მაშინ სრულდება, როცა მომხმარებელი ბეჭდვას 300ms-ით შეწყვეტს. ველში ტექსტი კი მაშინვე ჩანს.

11. მოდალური ფანჯრები (createPortal)

`ReactDOM.createPortal` მოდალს იმ ადგილას კი არ ხატავს, სადაც კომპონენტი დგას, არამედ პირდაპირ `document.body`-ში. ასე მას მშობელი ელემენტის `overflow: hidden` არ ჭრის და მშობლის `z-index`-ის კონტექსტი არ ზღუდავს: მოდალი ყოველთვის ყველაფრის თავზეა. React-ის ლოგიკა (state, props) მაინც ნორმალურად მუშაობს. ჩემი მოდალი იხურება ✕-ით, ფონზე დაჭერით და Escape-ით, ფოკუსიც სწორად იმართება.

12. Fail-Fast ვალიდაცია და Double-Click დაცვა

Fail-Fast ნიშნავს, რომ არასწორი მონაცემი შეცდომას მაშინვე, ფრონტენდზე აჩენს, სერვერზე გაგზავნამდე. ჩემთან Zod ამოწმებს SKU-ს ფორმატს, 13-ციფრიან შტრიხკოდს, პაროლის წესებს და იმას, რომ ჩამოსაწერი რაოდენობა ნაშთზე მეტი არ იყოს. `disabled={isSubmitting}` გაგზავნის დროს ღილაკს ბლოკავს (წერია „ინახება...“), ამიტომ სწრაფი ორმაგი დაჭერა ორ მოთხოვნას ვერ გაგზავნის და დუბლიკატი ჩანაწერი ან ორმაგი ჩამოწერა არ მოხდება.

13. Responsive Web Design

1440px-ზე Sidebar მუდმივად ჩანს და ცხრილი სრულად ჩანს. 768px-ზე Sidebar იმალება, ცხრილი ბარათებად იქცევა და ფილტრები ერთ ხაზზეა. 375px-ზე ერთსვეტიანი ბადეა და სტატისტიკა 2×2-ია, მოდალი კი მთელ ეკრანზეა. მობილურ ნავიგაციას ☰ ჰამბურგერ მენიუ ემსახურება: Sidebar გამოდის მარცხნიდან (`transform: translateX`), ფონი მუქდება, Escape ან ფონზე დაჭერა ხურავს. ტესტირება ჩავატარე 3 ბრაუზერში და 6 მოწყობილობაზე (90 კომბინაცია).

14. მდგრადობა და Debugging

ErrorBoundary Runtime შეცდომას იჭერს და თეთრი ეკრანის ნაცვლად აჩვენებს Fallback-ს ღილაკით „სცადეთ ხელახლა“, ხოლო Header და მენიუ აგრძელებს მუშაობას. Offline Fallback: თუ სერვერი 500-ს აბრუნებს ან ინტერნეტი გაწყდა, აპი აჩვენებს localStorage-ში შენახულ (ან დემო) მონაცემებს და ბანერს „⚠ ნაჩვენებია ლოკალური მონაცემები“. DevTools-ში ვიყენებ Breakpoint-ებს (Call Stack, Scope), `console.table`-ს (ნივთების სია ცხრილად), `console.time`-ს (ჩატვირთვის დრო) და Network ტაბს. ამით ვიპოვე მაგალითად Race Condition, როცა ძველი პასუხი ახალს გადაფარავდა.

ბლოკი V: წარმადობა, State Management და ტესტირება

15. Memoization და Code Splitting

`useMemo` მძიმე გამოთვლის შედეგს იმახსოვრებს (ჩემთან ფილტრაცია, სორტირება, სტატისტიკა) და ხელახლა ითვლის მხოლოდ მონაცემის შეცვლისას. `useCallback` ფუნქციას ყოველ რენდერზე თავიდან არ ქმნის, `React.memo` კი კომპონენტს ხელახლა არ ხატავს, თუ props არ შეცვლილა. ამიტომ ცხრილის 12 რიგიდან რედაქტირებისას მხოლოდ 1 იხატება. `React.lazy` + `Suspense` თითო გვერდს ცალკე ფაილად ტვირთავს მაშინ, როცა მასზე გადავდივართ: მთავარი JS ფაილი 429 kB-დან 267 kB-მდე შემცირდა (–38%).

16. მედიაფაილების ოპტიმიზაცია

WebP და AVIF იმავე ხარისხზე JPEG-სა და PNG-ზე ბევრად პატარაა: ჩემს პროექტში 4 სურათი PNG-ში 8.3 MB იყო, WebP-ში (480×360) კი 8.9 KB. `loading="lazy"` სურათს მხოლოდ მაშინ ტვირთავს, როცა ის ეკრანთან ახლოსაა, ამიტომ გვერდი უფრო სწრაფად იხსნება. `width`/`height` კი ჩატვირთვისას „ხტომას“ (CLS) აცილებს. იკონებისთვის SVG-ს ვიყენებ (Lucide React): ვექტორულია, ნებისმიერ ზომაზე მკვეთრია, მსუბუქია და ფერი CSS-ით იცვლება.

17. გლობალური სთეით-მენეჯმენტი და Persistence

Prop Drilling არის მონაცემის გადაცემა მშობლიდან შვილზე 3–4 დონეზე, მაშინაც კი, როცა შუალედურ კომპონენტებს ეს მონაცემი არ სჭირდებათ. ეს კოდს ართულებს და ცვლილებებს რთულს ხდის. გადავჭერი გლობალური Store-ით (**Context API + useReducer**, 11 action): Header, Dashboard, ცხრილი და მოდალი ერთსა და იმავე მონაცემს `useInventory()`-ით კითხულობენ. ამიტომ მოდალში ნივთის დამატებისას Header-ის მრიცხველი მაშინვე 12-დან 13-მდე იზრდება. Store-ის საწყისი მნიშვნელობა `localStorage`-იდან იკითხება, ყოველი ცვლილება კი `useEffect`-ით ინახება, ამიტომ Refresh-ის შემდეგ რჩება როგორც ნივთები, ისე ფილტრები.

ბლოკი VI: დოკუმენტაცია, ანგარიშგება და პრეზენტაცია

18. Cross-Browser ტესტირება და კოდის ჰიგიენა

ბრაუზერებს სხვადასხვა ძრავი აქვთ (Chrome/Edge Blink-ზეა, Firefox Gecko-ზე), ამიტომ ერთი და იგივე კოდი სხვადასხვაგვარად შეიძლება მუშაობდეს. ჩემს პროექტში აღმოჩნდა, რომ ქართული სორტირება Chrome-სა და Edge-ში განსხვავდებოდა (Chrome-ს ქართული ენის წესები არ აქვს) და ეს ცალკე ფუნქციით გავასწორე. `.gitignore` GitHub-ზე არ ტვირთავს `node_modules`-ს (მძიმეა, `npm install`-ით აღდგება), `dist`-ს (build-ის შედეგი) და `.env`-ს (კონფიდენციალური გასაღებები და პაროლები).

19. README.md vs ტექნიკური ანგარიში

README.md დეველოპერისა და შემფასებლისთვისაა: როგორ გაიგოს და გაუშვას პროექტი. სავალდებულო სექციებია: სათაური და Live Demo, ბიზნეს-აღწერა, ძირითადი ფუნქციონალი, ტექნოლოგიების ცხრილი, საქაღალდეების სტრუქტურა, გაშვების ინსტრუქცია (`git clone → npm install → npm run dev`), სატესტო ანგარიშები და სკრიპტები. ტექნიკური ანგარიში კი შესრულებულ სამუშაოს ასაბუთებს შეფასების კრიტერიუმების მიხედვით: დაგეგმვა, რისკები, გადაწყვეტილებების დასაბუთება, გადაჭრილი პრობლემები, ტესტირების შედეგები და Roadmap.

20. Live Demo და „Demo Effect“

5-წუთიანი დემო უნდა მიჰყვებოდეს ერთ ლოგიკურ Happy Path-ს. ჩემი სცენარია: ოპერატორით შესვლა (შეზღუდული მენიუ) → მენეჯერით შესვლა → ძიება და ფილტრი → ნივთის დამატება მოდალით (ჯერ ვალიდაციის შეცდომები, მერე შენახვა და Header +1) → Refresh (მონაცემი დარჩა) → ჩამოწერა ნაშთზე მეტის (დაცვა) → მობილური ხედი → ErrorBoundary. „Demo Effect“-ის თავიდან ასაცილებლად წინასწარ ვამოწმებ საიტს და ორივე ვარიანტს: Live Demo-ს (Vercel) და ლოკალურს (`npm run dev`). მზად მაქვს სატესტო ანგარიშები და სცენარი, მაქვს სქრინები და PDF ანგარიში სარეზერვოდ, ხოლო ავტომატური E2E ტესტით (18 შემოწმება) ვრწმუნდები, რომ ყველაფერი მუშაობს.